

Practical 5: State Management

State management refers to the management and manipulation of data within an app to ensure that the user interface (UI) accurately reflects the current state of the application. This lab utilizes the Provider package, which allows you to hold application's state.

1. We are going to create a mobile app that utilizing the **Provider** state management as below:

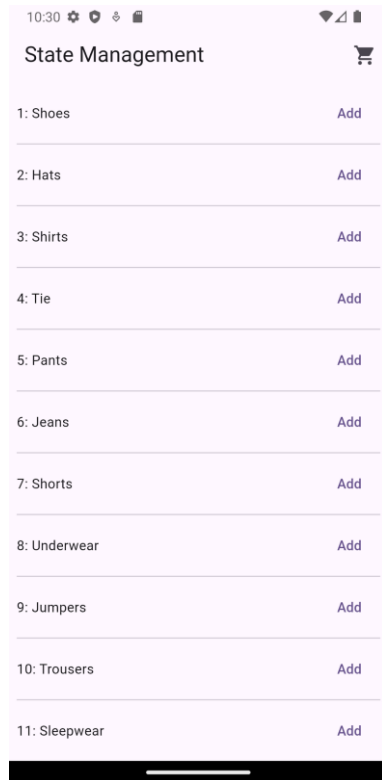


Figure 1 Item List

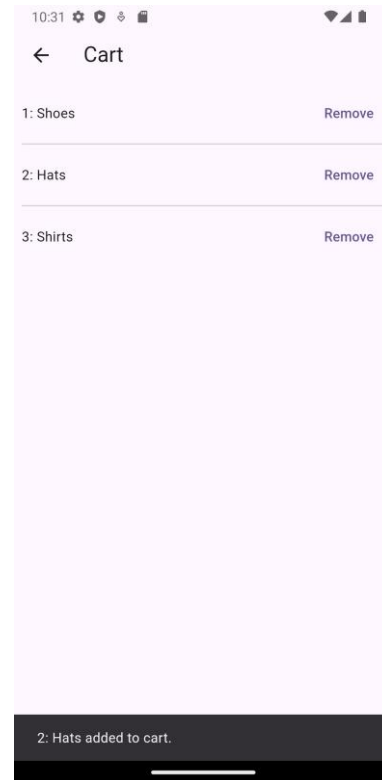


Figure 2 Cart

1. Create a new Flutter project using the **Application** template.
2. Open the **Terminal** and enter the following command:

```
flutter pub add provider
```

3. Create a new dart file named **item.dart**. This is a model for items to be stored in the memory.

```
class Item{
  int id;
  String name;
  //Constructor with initializing formal parameters
  Item(this.id, this.name);
  int get itemID => id;
  String get itemName => name;
  @override
  String toString() {
    return '$id: $name';
  }
}
```

4. Create a new dart file named **cart_provider.dart**. This class holds a list of items (the cart) that could be accessed by all the screens/routes within the app.

```
import 'package:flutter/material.dart';
import 'item.dart';

//Providing change notification to its listeners
class CartProvider extends ChangeNotifier {
  final List<Item> itemList = [];

  void add(Item item){
    itemList.add(item);
    notifyListeners();
  }

  void remove(Item item){
    itemList.remove(item);
    notifyListeners();
  }
}
```

5. In the **main.dart** file, observe the dummy data list as follows: -

```
final List<Item> catalog = [
  Item(1, 'Shoes'),
  Item(2, 'Hats'),
  Item(3, 'Shirts'),
  Item(4, 'Tie'),
  Item(5, 'Pants'),
  Item(6, 'Jeans'),
  Item(7, 'Shorts'),
  Item(8, 'Underwear'),
  Item(9, 'Jumpers'),
  Item(10, 'Trousers'),
  Item(11, 'Sleepwear'),
  Item(12, 'Accessories'),
];
```

This catalog list should be declared as a global variable, which can be above the main() function.

6. In the **main.dart** file, update the main() function to include an instance of a **ChangeNotifier** as follows:-

```
void main() {
  //TODO - The ChangeNotifierProvider provides an instance of a ChangeNotifier to
  its descendants
  runApp(
    ChangeNotifierProvider(
      create: (context) => CartProvider(),
      child: const MyApp(),
    ),
  );
}
```

7. In the **main.dart** file, insert a new class named **CartItem**, which is used to present an item in a list. Each cart item has an id, name, and an Add button:

1: Shoes	Add
----------	-----

```

class CartItem extends StatelessWidget {
  final int index;

  const CartItem({super.key, required this.index});

  @override
  Widget build(BuildContext context) {
    var item = catalog[index];
    return Row(
      children: [
        //TODO - Insert widgets for each row
        Text('$item'),
        const Expanded(child: SizedBox()),
        Consumer<CartProvider>(builder: (context, cart, child) {
          return TextButton(
            onPressed: () {
              cart.add(item);
              ScaffoldMessenger.of(context).showSnackBar(
                SnackBar(
                  content: Text('$item added to cart.'),
                ),
              );
            },
            child: const Text('Add'));
        }),
      ],
    );
  }
}

```

The body section contains a **Consumer** that can access a **ChangeNotifier** named **CartProvider**.

8. Modify the body section of the **build()** method of the **_MyHomePageState** to include a **ListView** as follows:-

```

body: Center(
  child: ListView.separated(
    padding: const EdgeInsets.all(8),
    scrollDirection: Axis.vertical,
    shrinkWrap: true,
    itemBuilder: (BuildContext context, int index) {
      return CartItem(index: index);
    },
    separatorBuilder: (BuildContext context, int index) =>
      const Divider(),
    itemCount: catalog.length
  ),
),

```

9. Run the project. You should be able to see a **ListView** as shown in Figure 1 Item List.
10. In the **main.dart** file, **MyAppState's build()** method, include an icon button in the **appBar** section:

```

@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: const Text('State Management'),
      actions: [
        IconButton(

```

```
onPressed: () {  
  Navigator.push<void>(  
    context,  
    MaterialPageRoute<void>(  
      builder: (BuildContext context) => const CartPage()));  
},  
icon: const Icon(Icons.shopping_cart),  
),  
],  
),
```

Ignore the error for CartPage(), we will create it later.

11. Create a new dart file named **card_dart** as follows: -

```
import 'package:demo_state_mgmt/cart_provider.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import 'item.dart';

class CartPage extends StatelessWidget {
  const CartPage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Cart'),
      ),
      body: const Column(
        mainAxisAlignment: MainAxisAlignment.start,
        children: [
          CartListWidget(),
        ],
      ),
    );
  }
}

class CartListWidget extends StatefulWidget {
  const CartListWidget({super.key});

  @override
  State<CartListWidget> createState() => _CartListWidgetState();
}

class _CartListWidgetState extends State<CartListWidget> {
  @override
  Widget build(BuildContext context) {
    return Consumer<CartProvider>(
      builder: (context, cart, child) {
        return cart.itemList.isNotEmpty
          ? ListView.separated(
              padding: const EdgeInsets.all(8),
              scrollDirection: Axis.vertical,
              shrinkWrap: true,
              itemBuilder: (BuildContext context, int index){
                return CartItem(item: cart.itemList[index]);
              },
              itemCount: cart.itemList.length,
              separatorBuilder: (BuildContext context, int index) => const
                Divider(),
            )
          : const Center(child: Text('Cart is empty.'));
      },
    );
  }
}
```

```

    }
  );
}
}

class CartItem extends StatelessWidget {
  final Item item;

  const CartItem({super.key, required this.item});

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text('$item'),
        const Expanded(child: SizedBox()),
        TextButton(
          onPressed: () {
            Provider.of<CartProvider>(context, listen: false).remove(item);
          },
          child: const Text('Remove')),
      ],
    );
  }
}

```

This widget is similar to the main.dart's MyApp() widget, however, the item list has a remove button.

12. Save and run the project. Try adding some items to the shopping list and then click the cart button.



TODO: Enhance the app by including a cart item counter displayed at the menu bar.